

CS:APP Data Lab 实验报告

整数位运算与 IEEE 754 浮点数操作

一、实验目标

本实验通过完成 P1–P19 共 19 个位级编程题目，理解 32 位整数的补码表示、位运算、溢出判断，以及 IEEE 754 单精度浮点数的编码与舍入规则。实验要求在各题规定的合法运算符、常量范围和操作数数量限制内实现函数，并通过规则检查与功能测试。

二、各题实现思路

P1signMask：利用左移将整数 1 移至最高位，构造最高位为 1、其余位为 0 的掩码。

P2bitXor：利用德摩根定律，将异或表达式转换为按位取反和按位与的组合，避免直接使用异或运算符。

P3negativePart：利用算术右移得到输入的符号掩码，并结合取反、加一构造相反数；再通过符号掩码筛选结果，使非负输入返回 0。

P4copyByteWithin：根据源字节和目标字节的位置计算移位置，提取源字节；清除目标字节原有内容后，将源字节移至目标位置并合并。

P5logicalShift：对有符号整数右移后，利用掩码清除算术右移产生的高位符号扩展，从而得到逻辑右移结果。

P6swapNibblePairs：使用掩码提取每个字节的高、低半字节，分别移位后交换位置，再将两部分合并。

P7secondLowestZeroBit：通过取反及补码运算定位最低位的零位，将该位置置为 1 后，再以相同思路定位第二个最低位的零位。

P8oddParity：通过多轮异或和右移，将不同位段的奇偶信息逐步折叠到低位，最终依据最低位判断 1 的个数是奇数还是偶数。

P9rotateRightBits：将移位置限制在有效范围内，分别计算右移部分和循环补回的左移部分，使用掩码去除无效位后合并结果。

P10roundEvenPow2：用掩码分离保留部分与待舍弃部分，比较余数和舍入中点；若恰好位于中点，则根据保留部分的奇偶性决定是否进位，实现向偶数舍入。

P11midpointTowardFirst：利用按位与和异或拆解中点计算，避免直接将两个整数相加造成溢出；再结合符号及低位信息，判断中点按题目要求向哪一端取整。

P12isBetweenEitherOrder：先确定两个端点的大小关系，再判断目标值是否位于下界与上界之间。为避免直接相减导致溢出，结合符号位和差值符号判断大小关系。

P13mul5Sat：使用左移和加法计算 $x \times 5$ ，再通过输入符号及其与正、负边界值的关系判断是否溢出。若溢出则返回对应饱和值，否则返回乘积。

P14classifyAdd3：分两步计算三个整数的和，并根据每次加法中操作数与结果的符号位变化判断溢出方向，综合判断数学意义上的总和是否超出有符号整数范围。

P15floatScaleThreeHalves：将输入视为 IEEE754 单精度浮点数的 32 位编码，提取符号位、阶码和尾数。对无穷大、NaN 等特殊值单独处理；普通数值则根据规格化数或非规格化数的特点调整有效数和阶码，并处理舍入及指数溢出。

P16floatRoundEven：根据阶码判断数值范围。对绝对值小于 1 的数按舍入规则处理，对本身已为整数的数保留原编码；其余情况分离整数部分和待舍弃部分，比较中点并执行向偶数舍入，最后处理有效数进位并组合结果。

P17float_i2f：先处理输入符号并取得整数绝对值，再寻找最高有效位以确定阶码。随后将有效数对齐到目标精度，根据被舍弃位判断是否需要舍入；若舍入引起有效数进位，则同步调整阶码，最后组合符号位、阶码和尾数。

P18bitCount：采用分组统计方法，先统计相邻位，再统计更大位段中的 1 的数量。通过掩码、移位和加法逐级合并计数，得到整个 32 位整数中 1 的总数。

P19bitReverse：通过多轮分组交换反转位序：依次交换相邻位、2 位组、4 位组、8 位组，最后交换高低 16 位。各轮利用掩码、移位和按位或完成。

三、测试结果

本次作业代码已提交至 GitHub 仓库。GitHubActions 自动评分页面显示 totalPoints 为 110、maxPoints 为 110，最终得分为 110/110，达到满分。

仓库地址：<https://github.com/A-fool-sleeping/Lab1>

本次提交：

<https://github.com/A-fool-sleeping/Lab1/commit/e0a75ba0feefe1c222045085051b9810750a2650>

按课程实验说明补充终端测试截图：运行 ./dlc-ebitsc 后的操作符检查结果，以及运行 ./btest 后的功能测试结果。请将本人实际运行得到的截图放入仓库 images/ 目录，并在提交前插入本节。

```

● jcc@LAPTOP-Q6JEH4VM:~/Lab1$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 18/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 15/16 operators
roundEvenPow2: 22/24 operators
midpointTowardFirst: 25/32 operators
isBetweenEitherOrder: 47/48 operators
mul5Sat: 29/30 operators
classifyAdd3: 39/52 operators
floatScaleThreeHalves: 18/60 operators
floatRoundEven: 23/65 operators
float_i2f: 23/40 operators
bitCount: 24/40 operators
bitReverse: 28/34 operators
All 19 functions passed operator checks.

```

```

● jcc@LAPTOP-Q6JEH4VM:~/Lab1$ ./btest
Score  Rating  Errors  Function
1      1      0      signMask
2      2      0      bitXor
3      3      0      negativePart
4      4      0      copyByteWithin
4      4      0      logicalShift
4      4      0      swapNibblePairs
4      4      0      secondLowestZeroBit
5      5      0      oddParity
5      5      0      rotateRightBits
5      5      0      roundEvenPow2
5      5      0      midpointTowardFirst
7      7      0      isBetweenEitherOrder
7      7      0      mul5Sat
7      7      0      classifyAdd3
7      7      0      floatScaleThreeHalves
10     10     0      floatRoundEven
10     10     0      float_i2f
10     10     0      bitCount
10     10     0      bitReverse
Total points: 110/110

```

四、实验总结

本实验让我进一步理解了补码、位掩码、移位和符号位判断等底层概念。整数题要求在严格限制下实现常见操作，使我学会将问题拆解为基本位运算；溢出题使我认识到需要区分机器运算结果与数学上的真实结果。浮点数题通过直接操作符号位、阶码和尾数，加深了我对 IEEE754 编码、特殊值及舍入规则的理解。

实验也体现了规则检查和功能测试的区别：前者验证代码是否符合限制，后者验证计算结果是否正确。只有同时满足编码规则与功能要求，才能完成实验目标。